Tech Challenge - Sistema de Agendamento Hospitalar (Microsserviços)

Equipe: Indivudal

Integrantes: Erick Calazães (RM 368842)

Link do Repositório (GitHub): https://github.com/ecalazaes/techchallenge-backend-3

Este projeto faz parte da Fase 3 do Tech Challenge da Pós-Graduação em Desenvolvimento Backend. Trata-se de uma arquitetura baseada em microsserviços para gestão de agendamentos médicos, processamento de histórico e notificações em tempo real.

🛠️ Tecnologias Utilizadas

- **Java 17**
- **Spring Boot 3.x**
- **Spring Data JPA**
- **Spring Security & JWT** (Controle de acesso e autenticação)
- **RabbitMQ** (Mensageria assíncrona entre serviços)
- **PostgreSQL** (Bancos de dados independentes por serviço)
- **Docker & Docker Compose** (Orquestração de containers)
- **SpringDoc OpenAPI (Swagger)**
- **Maven**

🏛️ Arquitetura e Design do Sistema

O projeto foi desenhado utilizando o padrão de **Microsserviços**, onde cada domínio possui seu próprio banco de dados e responsabilidades isoladas.

- **Scheduling Service (Porta 8081)**: Responsável pela criação e gestão de agendamentos. Atua como o produtor de eventos.
- **Notification Service (Porta 8082)**: Consome eventos de agendamento para disparar comunicações.
- **History Service (Porta 8083)**: Consome eventos para manter um registro histórico e auditoria. Possui interface **GraphQL**.

Padrões Aplicados:

- **Database per Service**: Isolamento total de dados para garantir independência de deploy.
- **Event-Driven Architecture**: Comunicação via RabbitMQ (Fanout Exchange).
- **Resiliência**: Implementação de **Retry Policy** e **Dead Letter Queues (DLQ)**.
- **Global Exception Handling**: Padronização de erros em todos os serviços.

🔐 Segurança e Controle de Acesso (JWT)

A segurança é transversal aos microsserviços, garantindo que apenas requisições autenticadas e autorizadas sejam processadas:

- **Autenticação Stateless**: Utilização de **JSON Web Tokens (JWT)** para manter a sessão do usuário sem estado no servidor.
- **Spring Security Filter Chain**: Cada serviço valida a integridade e a expiração do token em cada requisição.
- **RBAC (Role-Based Access Control)**: Diferenciação de permissões entre PACIENTE, MEDICO e ENFERMEIRO.
- **Proteção de Endpoints**:
  - Criação de agendamentos e consultas de histórico exigem o header Authorization: Bearer <token>.
  - Documentações (Swagger/GraphiQL) configuradas para acesso facilitado em ambiente de desenvolvimento.
  -

🐳 Como Executar (Docker)

A infraestrutura completa (serviços, RabbitMQ e Bancos) está configurada via Docker Compose.

1. Certifique-se de ter o **Docker** instalado.
2. Na raiz do projeto, execute o comando para build e subida:

   ```
   docker-compose up --build
   ```

3. **Bancos de Dados**: O sistema cria automaticamente o scheduling_db e executa o script init.sql para criar os schemas notification_db e history_db.

📖 Documentação da API (Swagger & GraphQL)

Acesse as interfaces enquanto os containers estiverem rodando:

- **Scheduling API (Swagger)**: http://localhost:8081/swagger-ui.html
- **History (GraphiQL)**: http://localhost:8083/graphiql
- **RabbitMQ Dashboard**: http://localhost:15672

🚀 Como Testar (Postman)

Dentro da pasta /postman na raiz do projeto, importe a collection: TechChallenge_Fase3_Collection.json.

Fluxo de Testes e Endpoints

1. Login e Autenticação Realize login e utilize o JWT token retornado no Header Authorization: Bearer <token> nas próximas requisições. * **POST** http://localhost:8081/api/auth/login * *Usuários de teste:* medico_erick / enfermeiro_gabriel / paciente_dani / paciente_gw2 | *Senha:* 123

```
{
  "username": "medico_erick",
  "password": "123"
}
```

2. Criação e Gestão de Consultas (Scheduling Service - REST) Criar Consulta (POST) http://localhost:8081/api/appointments

```
{
  "patientName": "Daniele Pinheiro",
  "patientUsername": "paciente_dani",
  "patientEmail": "daniele@teste.com",
  "doctorName": "Dr. Silva",
  "appointmentDate": "2026-03-25T15:03:00",
  "status": "SCHEDULED"
}
```

Editar Consulta (PUT) http://localhost:8081/api/appointments/1

```
{
  "doctorName": "Dr. Silva",
  "appointmentDate": "2026-05-25T15:30:00",
  "status": "UPDATED_SCHEDULED"
}
```

Deletar Consulta (DELETE) http://localhost:8081/api/appointments/1

3. Auditoria e Histórico Clínico (History Service - GraphQL) * POST
http://localhost:8083/graphql

Retornar Consultas Gerais:

```
query {
  getAllHistories {
    id
    patientEmail
    doctorName
    appointmentDate
    status
    notes
  }
}
```

Retornar Consultas Futuras por E-mail:

```
query {
  getHistoryByEmail(patientEmail: "daniele@teste.com", futureOnly: true) {
    appointmentDate
    doctorName
    status
  }
}
```

Adicionar Notas (Restrito a Médico ou Enfermeiro):

```
mutation {
  addNotesToHistory(id: "1", notes: "Paciente apresenta melhora no quadro clínico. Recomenda-se repouso.") {
    id
    patientEmail
    notes
  }
}
```

🛡️ Regras de Negócio & Resiliência

1. Comunicação Assíncrona e Fanout

Utilizamos uma **Fanout Exchange** (appointment.fanout). Quando um agendamento é criado, o Scheduling Service publica uma mensagem que é replicada automaticamente para as filas de Notificação e Histórico, garantindo desacoplamento total.

2. Estratégia de Tolerância a Falhas (Retry & DLQ)

Para garantir a resiliência: * **Retry Policy**: Cada consumidor tenta reprocessar mensagens falhas até 3 vezes com intervalos de 2 segundos. * **Dead Letter Queue (DLQ)**: Caso o erro persista, a mensagem é movida para uma fila de erro (.dlq) para posterior auditoria, evitando o travamento da fila principal.

3. Integridade e Validação

- **Bean Validation**: DTOs protegidos com @NotBlank, @Email e @Future, garantindo a qualidade dos dados de entrada.
- **Tratamento Global de Exceções**: Implementamos um @ControllerAdvice que mapeia erros para códigos HTTP semânticos (400, 404, 409, 500).